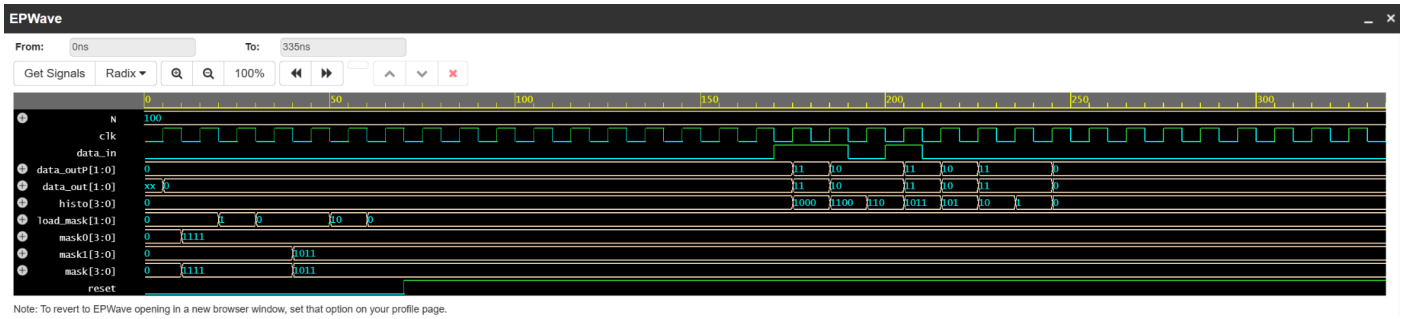
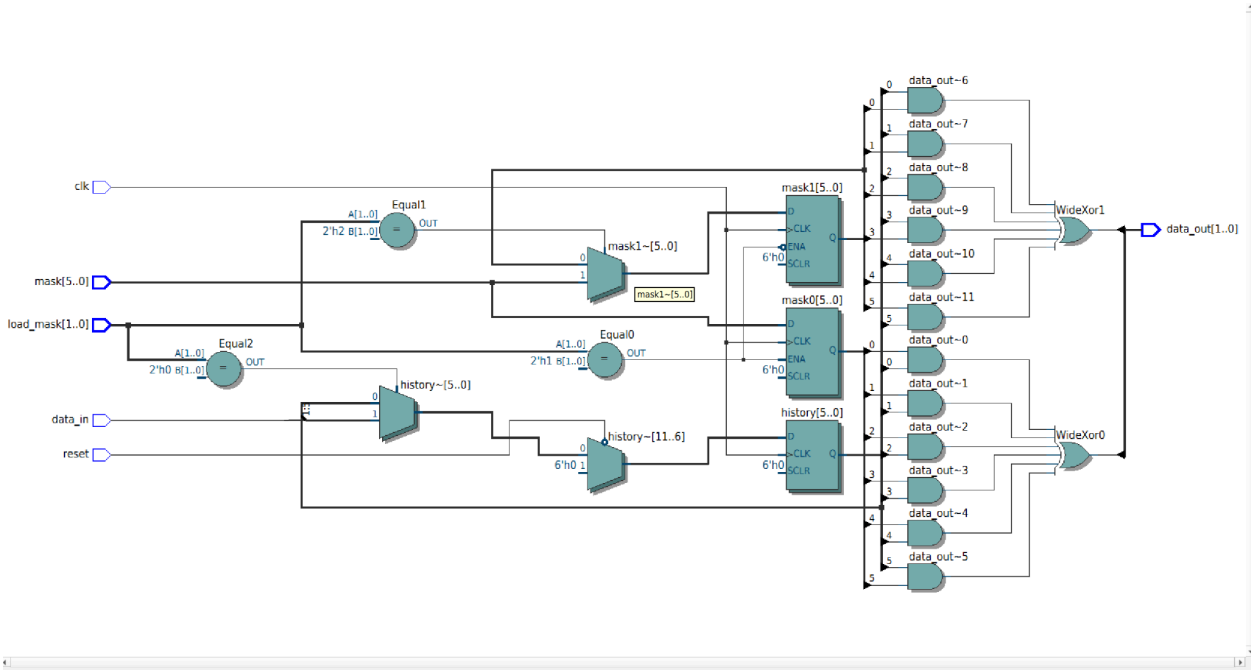


Above we can see that the output signal only returns what we want and wen we want it based on the control signals in the input. This shows the desired behavior we wanted to see for this module, correctly demonstrating the use cases it was designed for. Furthermore, you can see the output signal directly matches the testbench's desired output signal.

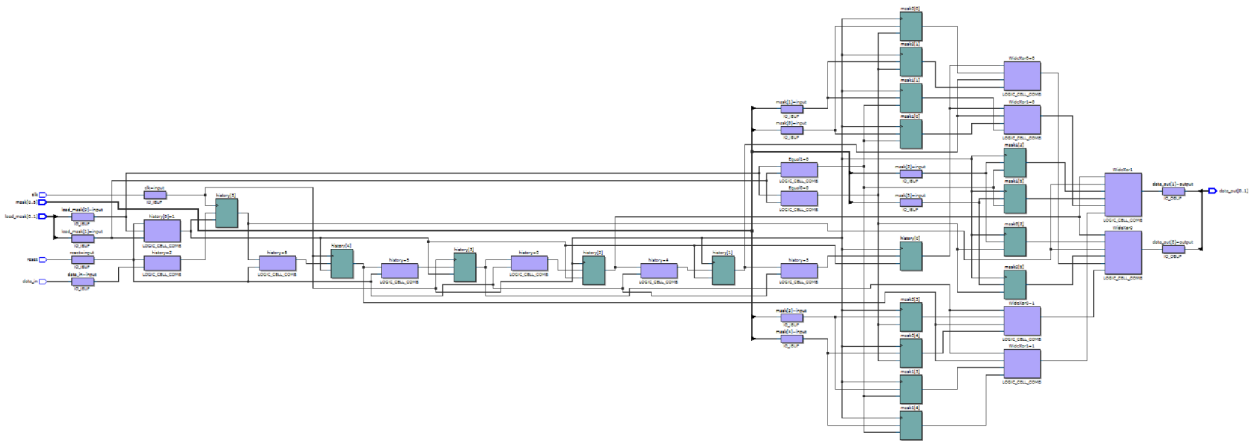
```
# run -all
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 11, got 11  YAA!
# expected 10, got 10  YAA!
# expected 10, got 10  YAA!
# expected 11, got 11  YAA!
# expected 10, got 10  YAA!
# expected 11, got 11  YAA!
# expected 11, got 11  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
# expected 00, got 00  YAA!
```

YAA!

Above is the transcript from the log printed from the testbench where we can clearly see the module passing every test given in the testbench, showing that it has the desired behavior.



	Resource	Usage
1	Estimate of Logic utilization (ALMs needed)	12
2		
3	▼ Combinational ALUT usage for logic	15
1	-- 7 input functions	0
2	-- 6 input functions	2
3	-- 5 input functions	0
4	-- 4 input functions	4
5	-- <=3 input functions	9
4		
5	Dedicated logic registers	18
6		
7	I/O pins	13
8		
9	Total DSP Blocks	0
10		
11	Maximum fan-out node	clk...put
12	Maximum fan-out	18
13	Total fan-out	116
14	Average fan-out	1.97



Module code (it said to put it here in the directions):

```
module conv_enc #(parameter N = 6)(
  input              clk,
  input              data_in,
  input              reset,
  input      [  1:0] load_mask,  // 01: load mask0; 10: load mask1
  input      [N-1:0] mask,
  output logic[  1:0] data_out
);

  // Internal Registers
  logic [N-1:0] mask0;
  logic [N-1:0] mask1;
  logic [N-1:0] history;


  always_ff @(posedge clk) begin
    if (load_mask == 2'b01)
      mask0 <= mask;
    else if (load_mask == 2'b10)
      mask1 <= mask;
  end

 always_ff @(posedge clk) begin
    if (!reset) begin
      history <= '0;
```

```
   end else if (load_mask == 2'b00) begin
     history <= {data_in, history[N-1:1]};
   end

  end


  always_comb begin
    data_out[0] = ^(mask0 & history);
    data_out[1] = ^(mask1 & history);
  end

endmodule

Tb:
module conv_enc_tb;

parameter N = 4;                  // set to desired constraint length + 1;
bit          clk, data_in, reset;
bit  [  1:0] load_mask;                  // 01: load mask 0; 10: load mask 1
bit  [N-1:0] mask, mask0, mask1,    // prepend desired mask vaue with 1
             histo;
logic[  1:0] data_outP;
wire [  1:0] data_out;                           // assumes rate 1/2

// rate 1/2, constraint N-1 convolutional encoder
conv_enc #(.N(N)) ce1(.clk,
             .data_in,
                                .reset,
                                .load_mask,
                                .mask,
                                .data_out);

always begin
  #5ns clk = 1'b1;
  #5ns clk = 1'b0;
end

always @(posedge clk)               // histo = "history"
  histo <= {data_in,histo[N-1:1]};         // just a shift register

always @(negedge clk)
  if(data_outP == data_out)
    $display("expected %b, got %b  YAA!",data_outP,data_out);
```

```
  else
    $display("expected %b, got %b  DAMN!",data_outP,data_out);

initial begin
  #10ns mask      =  'o17;//'o15;                    // 5 with 1 prepended
        mask0     =  mask;
  #10ns load_mask = 2'b01;
  #10ns load_mask = 2'b00;
  #10ns mask      =  'o13;//'o17;
        mask1     =  mask;
  #10ns load_mask = 2'b10;
  #10ns load_mask = 2'b00;
  #10ns reset     = 1'b1;                    // start running
  #10ns data_in   = 1'b0;                    // sequence from Viterbi demo
  #90ns data_in   = 1'b1;
// sequence from thesis
  #10ns data_in   = 1'b1;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b1;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b0;

  #40ns $stop;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b1;
  #10ns data_in   = 1'b1;
  #10ns data_in   = 1'b0;
  #40ns $stop;
  #40ns data_in   = 1'b1;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b1;
  #10ns data_in   = 1'b1;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b0;
```

```
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b0;
  #10ns $stop;
  #40ns data_in   = 1'b1;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b1;
  #10ns data_in   = 1'b1;
  #10ns data_in   = 1'b1;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b0;
  #40ns $stop;
  #10ns data_in   = 1'b1;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b1;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b0;
  #10ns data_in   = 1'b1;
  #10ns data_in   = 1'b0;
  #60ns $stop;
end

always_comb begin
  data_outP[1] = ^(mask1&histo);
  data_outP[0] = ^(mask0&histo);
end
  initial begin
    $dumpfile("dump.vcd"); $dumpvars;
  end

endmodule
```